

CS 221: Computer Programming II

Homework #5: Stack

Due: Monday, April 25 at 11:59 PM (Charleston Time) on OAKS.

Late Submissions will NOT be accepted. NO EXCEPTION!!

[100 points]: Maze Traversal Using Stacks

Goals: The second part of this assignment will prepare you for that day when you find yourself stuck on a deserted island and the only way out is through a maze. You will implement an efficient path finding algorithm to solve the maze search problem using a *stack*. The stack will help you keep track of valid (open) positions you have visited during the maze traversal. You are required to use Java's Stack class for this part. Refer to Java's official documentation here, if needed:

<https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html>.

Requirements: A maze is a square space with an entrance at the upper left-hand corner, an exit at the lower right-hand corner, and some internal walls. Your algorithm will find a path through a given maze, starting from the entrance and finishing at the exit that does not go through any walls (a valid path must go around walls).

Your program will take a command line argument, the filename (e.g. `maze1.txt`) for the maze description. (To input a command line argument in Eclipse or IntelliJ, you can click "Run" – "Run Configurations" – "Arguments" and then type in the argument in the "Program Arguments" box.) Your program next prints the maze to stdout, then tries to find a path through the maze. If you are using an IDE, please make sure that you have the text files directly under the main folder of your project.

If a path is found, the maze should be printed while indicating which positions were visited during the search, and a success message should be shown: **Maze is solvable**. Otherwise, an error message is printed: **Maze is NOT solvable**.

The maze will be represented as an $N \times N$ array of 1s and 0s; if `maze[i][j] == 1` then there is an internal wall in that position of the maze. Otherwise, there is no wall. The search algorithm will start at `maze[0][0]` and find a path to `maze[N-1][N-1]`. A path is represented by a sequence of `[i][j]` position coordinates, starting with `[0][0]` and ending with `[N-1][N-1]`. From `position[i][j]` in the path, the next position in the path can only be the position to the left, right, up or down from `position[i][j]`; a path cannot move diagonally from one position to another. For example, the following is the array representation of a 10×10 maze:

```
ENTER --> 0  1  1  1  0  0  0  0  0  0
          0  0  0  1  0  0  0  1  0  0
          0  1  0  1  1  0  0  1  0  0
          0  1  0  0  1  0  1  1  0  0
          0  1  0  0  1  0  1  1  0  0
          1  1  1  0  1  0  1  0  0  0
          0  0  1  0  0  0  1  0  1  1
          0  0  1  0  0  0  1  0  1  1
          0  1  1  0  1  0  1  0  0  0
          0  0  0  0  1  0  1  1  0  0 --> EXIT
```

The following path is possible through this maze:

Path: (`[0][0]`, `[1][0]`, `[1][1]`, `[1][2]`, `[2][2]`, `[3][2]`, `[3][3]`, `[4][3]`, `[5][3]`, `[6][3]`, `[6][4]`, `[6][5]`, `[5][5]`, `[4][5]`, `[3][5]`, `[2][5]`, `[2][6]`, `[1][6]`, `[0][6]`, `[0][7]`, `[0][8]`, `[1][8]`, `[2][8]`, `[3][8]`, `[4][8]`, `[5][8]`, `[5][7]`, `[6][7]`, `[7][7]`, `[8][7]`, `[8][8]`, `[8][9]`, `[9][9]`)

```

ENTER  --> X    1    1    1    0    0    X----X----X    0
          |
          X----X----X    1    0    0    X    1    X    0
                |
0         1    X    1    1    X----X    1    X    0
                |
0         1    X----X    1    X    1    1    X    0
                |
0         1    0    X    1    X    1    1    X    0
                |
1         1    1    X    1    X    1    X----X    0
                |
0         0    1    X----X----X    1    X    1    1
                |
0         0    1    0    0    0    1    X    1    1
                |
0         1    1    0    1    0    1    X----X----X
                |
0         0    0    0    1    0    1    1    0    X --> EXIT

```

The Path Finding Algorithm: You should implement the following algorithm for finding a path to exit the maze (some of the details are left for you to fill in). You are required to use Java's Stack class for solving this assignment. You will use the stack to push neighbor (valid) positions and to pop the next position at each step. Think carefully about when a neighbor is 'valid' and how you can ensure that your implementation terminates.

Create a search list for positions yet to explore, add the entrance position (0,0) to the search

while the list is not empty **do**

 Remove the next position from the search list

if it is the exit position [n-1] [n-1] **then**

 an escape path was found, return a success message and print the traversed maze

else

 mark the position as visited, then push all valid up, down, left, or right neighbor positions to the search list

end if

end while

if the list is empty and the method has not returned **then**

 there is no path

end if

Data Structures: The maze is stored as a 2-dimensional array of char values defined as `char[][]`. You can see how to declare, initialize, and use 2-dimensional arrays in the provided method `readMaze` in class `MazeSolver`.

Additionally, a position in the maze can be represented as an object of class `MazePosition`. This simple class can be used to store the x and y coordinates of a position. As you walk through the maze, you should keep track of valid positions by creating a Java Stack (from `java.util.Stack`), then adding or removing `MazePosition` objects to it as needed.

Getting Started:

1. Download the starter code on OAKS and understand it. Do not modify class `MazePosition`. Class `MazeSolver` has the following methods that are already implemented:
 - `readMaze(String filename)`: reads maze from file, stores it in a 2-dimensional array with each entry containing a 0 or a 1 (wall).
 - `printMaze(char[] [] maze)`: prints a visual representation of the 2-dimensional maze array to stdout.
 - `main()`: reads and prints a maze, searches for an escape path using `solveMaze` (to be implemented by you), then prints the maze again after the traversal.
2. Fill in your implementation for the `solveMaze` method in class `MazeSolver`. Read and follow the comments in the starter code carefully. You are free to create additional helper methods if needed; e.g., if you find yourself repeating code to perform a certain functionality, that's a sign that you may need a method for that!
3. Test your program with the provided mazes (`maze1.txt`, `maze2.txt`, `maze3.txt`, `maze4.txt`).
4. Submit **only** `MazeSolver.java` for this assignment.

Sample Run: Here's an example of how a program run might look like. Note that the maze file name is passed as an argument to the program here using the command line:

```
$ java MazeSolver maze1.txt
[Before Traversal] Maze:
0 1 0 1 0
0 0 0 1 0
0 1 0 0 0
0 1 0 1 1
0 1 0 0 0

Maze is solvable.

[After Traversal] Maze:
X 1 0 1 0
X X X 1 0
X 1 X 0 0
X 1 X 1 1
X 1 X X X
```

Submission Instructions

Only submit the following java file on OAKS:

MazeSolver.java.

Grading

- If a program does not compile, you will get 0 points for it :-(
- The program successfully solves the maze (and detects if the maze is not solvable) using a stack algorithm [90 points]
- Overall code clarity, readability, and style [10 points]

Required program documentation

Header comment at the start of each file must include:

- Your name
- HW Number, Class Section Number, Spring 2022
- A one- to three-sentence description of the purpose of the code in this file

Honor Code

The assignment is governed by the College Honor Code and Departmental Policy. Remember, any code you submit must be your own; otherwise you risk being investigated by the Honor Council and facing the consequences of that. We do check for code plagiarism (across student groups and against online resources). Please remember to have the following comment included at the top of **ALL** java file.

```
/*  
THIS CODE WAS OUR OWN WORK, IT WAS WRITTEN WITHOUT CONSULTING  
CODE WRITTEN BY OTHER STUDENTS OR COPIED FROM ONLINE RESOURCES.  
_Student_Name_Here_ */
```

Very Important: Your method headers (i.e., return type, method name, and the list of parameters) should **EXACTLY** match the description in the handout and the starter code; otherwise, our testing scripts will fail and you risk getting 0 points.